

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Knowledge Representation Design
Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA01**

COURSE OUTCOME COVERED

CO3: *Implement propositional and first-order logic using resolution, unification, and inference techniques for knowledge representation and reasoning. (BTL 3)*

Assessment Tool Weightage: 40 %

Total Marks: 100

Time: 60 Mins

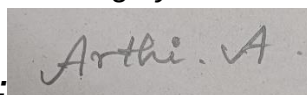
No. of Questions: 5

Annexure A
Knowledge Representation Design Assessment

Code of Conduct:

I ARTHI A (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Engineering and Knowledge Base Design	15%	
3. Propositional Logic Representation	10%	
4. First-Order Logic Representation	15%	
5. Unification and Resolution	15%	
6. Forward and Backward Chaining	15%	
7. Inference and Reasoning Accuracy	10%	
8. System Architecture / Representation Diagram	5%	
9. Prototype Implementation and Results	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Problem statement - Intelligent Vehicle Fault Diagnosis

An automobile service centre wants an expert system that identifies possible vehicle faults from symptoms. The domain rules state: a vehicle has a dead battery if it does not start and the headlights are dim; a vehicle may have a fuel problem if the engine cranks but does not start; a vehicle may have an overheating problem if the temperature is high and coolant is low; and a vehicle should be sent for immediate service if overheating is detected.

For a vehicle named Car101, the system observes: Car101 does not start, Car101 has dim headlights, and Car101 has a high temperature.

1. Knowledge representation design

The system is a rule-based expert system with three components:

- Knowledge base (KB): ground facts about a specific vehicle plus general domain rules (implications) that hold for any vehicle.
- Inference engine: applies forward chaining (data-driven) and backward chaining (goal-driven) over the KB, and can run resolution refutation to formally prove a query.
- Working memory: holds derived facts as they are generated during forward chaining, so later rules can fire on them.

Entities are vehicles (e.g. the constant Car101). Predicates capture observable symptoms (Starts, HeadlightsDim, Cranks, TempHigh, CoolantLow) and diagnostic conclusions (DeadBattery, FuelProblem, Overheating, NeedsService).

2. Propositional logic representation

For a fixed vehicle, let:

- N = “does not start” ($\neg$ Starts), H = “headlights are dim”, C = “engine cranks”
- T = “temperature is high”, L = “coolant is low”
- D = “dead battery”, F = “fuel problem”, O = “overheating”, S = “needs immediate service”

Rules:

- 1. $N \wedge H \rightarrow D$
- 2. $C \wedge N \rightarrow F$
- 3. $T \wedge L \rightarrow O$

- 4. $O \rightarrow S$
- 5. $D \rightarrow S$

Facts for Car101: N, H, T. (C and L are not asserted — this becomes important during chaining.) Propositional logic is fine for one fixed car, but cannot generalise across vehicles, which motivates the first-order representation below.

3. First-order logic representation

Predicates: Starts(x), HeadlightsDim(x), Cranks(x), TempHigh(x), CoolantLow(x), DeadBattery(x), FuelProblem(x), Overheating(x), NeedsService(x)

Rules (generalised over any vehicle x):

- 1. $\forall x (\neg \text{Starts}(x) \wedge \text{HeadlightsDim}(x)) \rightarrow \text{DeadBattery}(x)$
- 2. $\forall x (\text{Cranks}(x) \wedge \neg \text{Starts}(x)) \rightarrow \text{FuelProblem}(x)$
- 3. $\forall x (\text{TempHigh}(x) \wedge \text{CoolantLow}(x)) \rightarrow \text{Overheating}(x)$
- 4. $\forall x \text{Overheating}(x) \rightarrow \text{NeedsService}(x)$
- 5. $\forall x \text{DeadBattery}(x) \rightarrow \text{NeedsService}(x)$

Facts (Car101): $\neg \text{Starts}(\text{Car101})$, $\text{HeadlightsDim}(\text{Car101})$, $\text{TempHigh}(\text{Car101})$.

4. Unification

Unification matches a query against a rule's consequent by finding a substitution that makes the two expressions identical.

- Query $\text{NeedsService}(\text{Car101})$ vs. rule consequent $\text{NeedsService}(x) \rightarrow \text{MGU} = \{x/\text{Car101}\}$, grounding rule 5 to $\text{DeadBattery}(\text{Car101}) \rightarrow \text{NeedsService}(\text{Car101})$.
- Query $\text{DeadBattery}(\text{Car101})$ vs. rule consequent $\text{DeadBattery}(x)$ from rule 1 $\rightarrow \text{MGU} = \{x/\text{Car101}\}$, grounding rule 1 to $(\neg \text{Starts}(\text{Car101}) \wedge \text{HeadlightsDim}(\text{Car101})) \rightarrow \text{DeadBattery}(\text{Car101})$.

Both substitutions bind x to the same constant throughout, so unification succeeds and the grounded rules can be used directly with the known facts.

5. Forward chaining

Starting from the facts $\{\neg \text{Starts}(\text{Car101}), \text{HeadlightsDim}(\text{Car101}), \text{TempHigh}(\text{Car101})\}$:

- Rule 1 fires ($\neg \text{Starts} \wedge \text{HeadlightsDim}$ both true) $\rightarrow$ derive $\text{DeadBattery}(\text{Car101})$.

- Rule 5 fires (DeadBattery now true) → derive NeedsService(Car101).
- Rule 3 cannot fire: TempHigh(Car101) is true, but CoolantLow(Car101) is unknown, so Overheating(Car101) is not derived.
- Rule 2 cannot fire: Cranks(Car101) is unknown.

Conclusion: Car101 has a dead battery and needs service. The overheating hypothesis stays open pending a coolant-level reading.

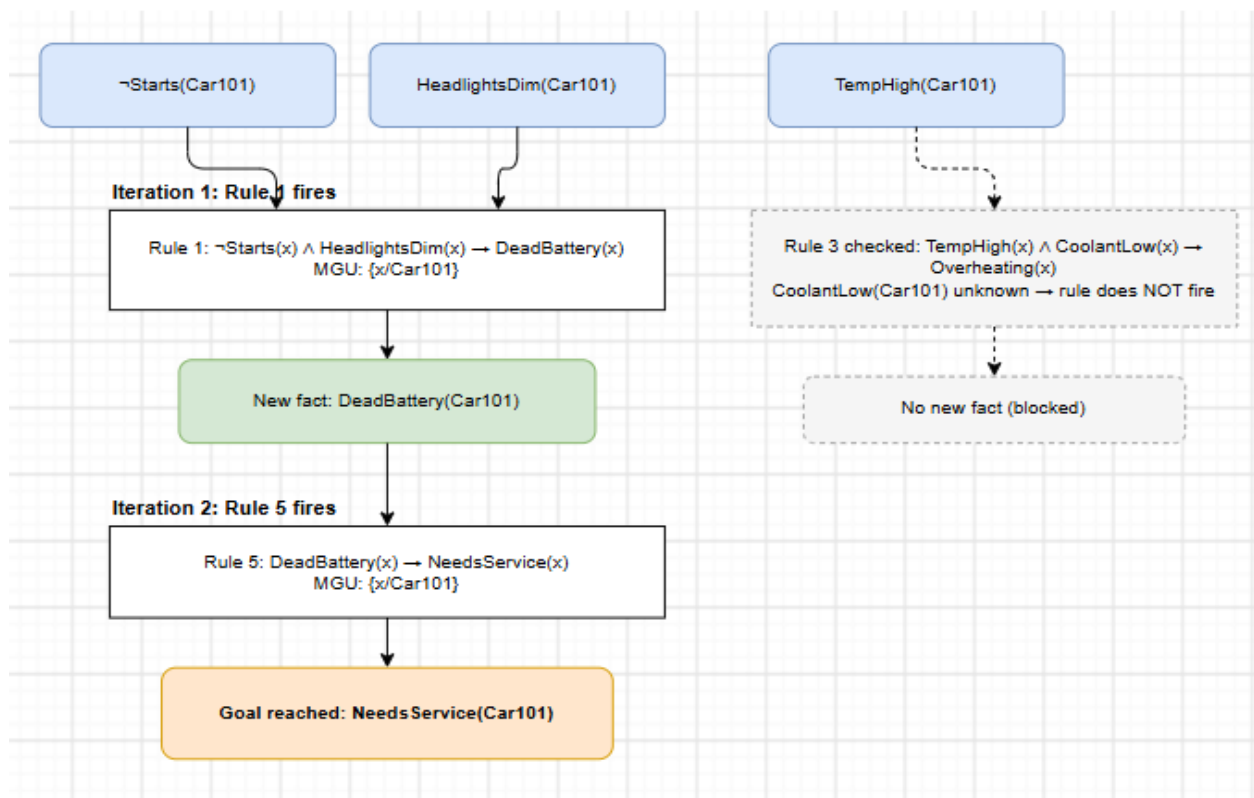


Figure 1 — Forward chaining trace

6. Backward chaining

Goal: prove `NeedsService(Car101)`.

- Via rule 4: need `Overheating(Car101)` → need `TempHigh(Car101)` (true) $\wedge$ `CoolantLow(Car101)` (unknown) → branch fails/unresolved.
- Via rule 5: need `DeadBattery(Car101)` → need `¬Starts(Car101)` (true) $\wedge$ `HeadlightsDim(Car101)` (true) → branch succeeds.

Since at least one branch succeeds, `NeedsService(Car101)` is proved true, via the dead-battery path.

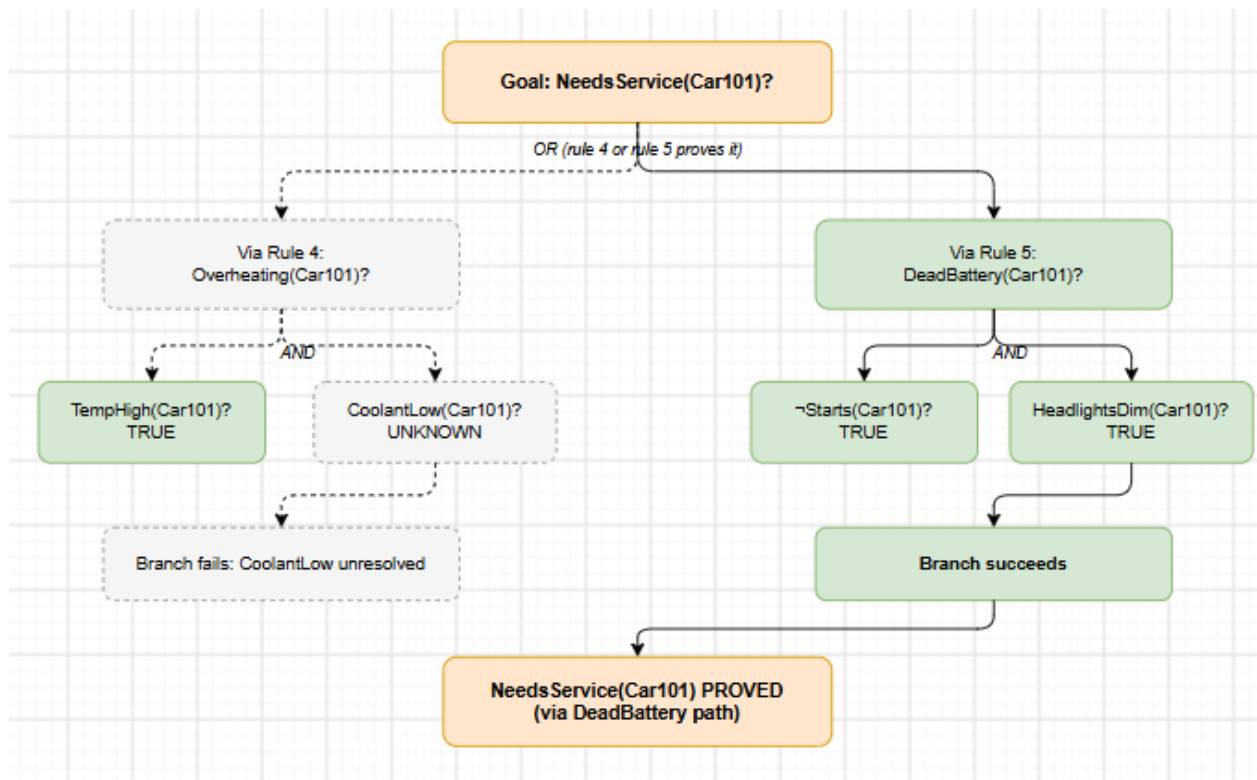


Figure 2 — Backward chaining goal tree

7. Resolution

To prove **DeadBattery(Car101)**, convert rule 1 to clausal form and resolve against the negated goal.

- Clause A (rule 1): $\text{Starts}(x) \vee \neg \text{HeadlightsDim}(x) \vee \text{DeadBattery}(x)$
- Clause B (fact): $\neg \text{Starts}(\text{Car101})$
- Clause C (fact): $\text{HeadlightsDim}(\text{Car101})$
- Clause D (negated goal): $\neg \text{DeadBattery}(\text{Car101})$

Resolve A with B (unify $x/\text{Car101}$, cancel $\text{Starts}(\text{Car101})/\neg \text{Starts}(\text{Car101})$) $\rightarrow$ Clause E: $\neg \text{HeadlightsDim}(\text{Car101}) \vee \text{DeadBattery}(\text{Car101})$.

Resolve E with C (cancel $\text{HeadlightsDim}(\text{Car101})$) $\rightarrow$ Clause F: $\text{DeadBattery}(\text{Car101})$.

Resolve F with D (cancel $\text{DeadBattery}(\text{Car101})/\neg \text{DeadBattery}(\text{Car101})$) $\rightarrow$ empty clause (contradiction) — the negated goal is refuted, so **DeadBattery(Car101)** is proved.

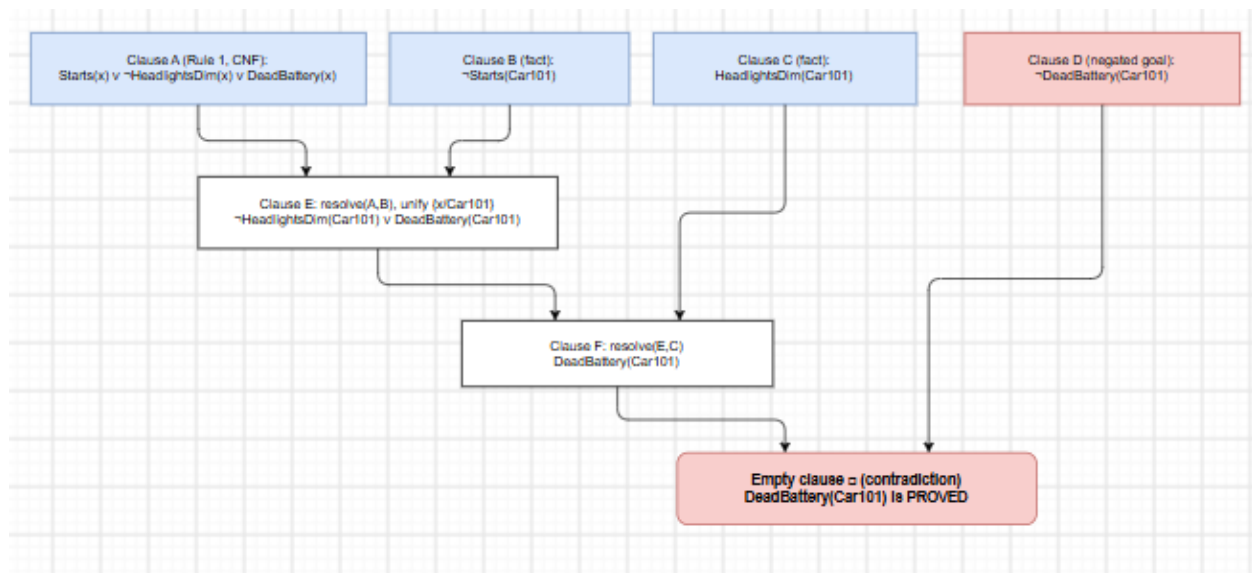


Figure 3 — Resolution refutation tree

8. Final inference

Combining all three methods, the system concludes that Car101 has a dead battery and requires service, based directly on the symptoms “does not start” and “dim headlights.” The overheating hypothesis is left unresolved rather than falsely confirmed, because the coolant-level fact was never asserted — the KB correctly distinguishes “false” from “unknown.” This shows a key strength of rule-based diagnosis: conclusions are only as strong as the facts fed in, and the inference engine is explicit about what it can and cannot yet determine.

System architecture / knowledge representation diagram

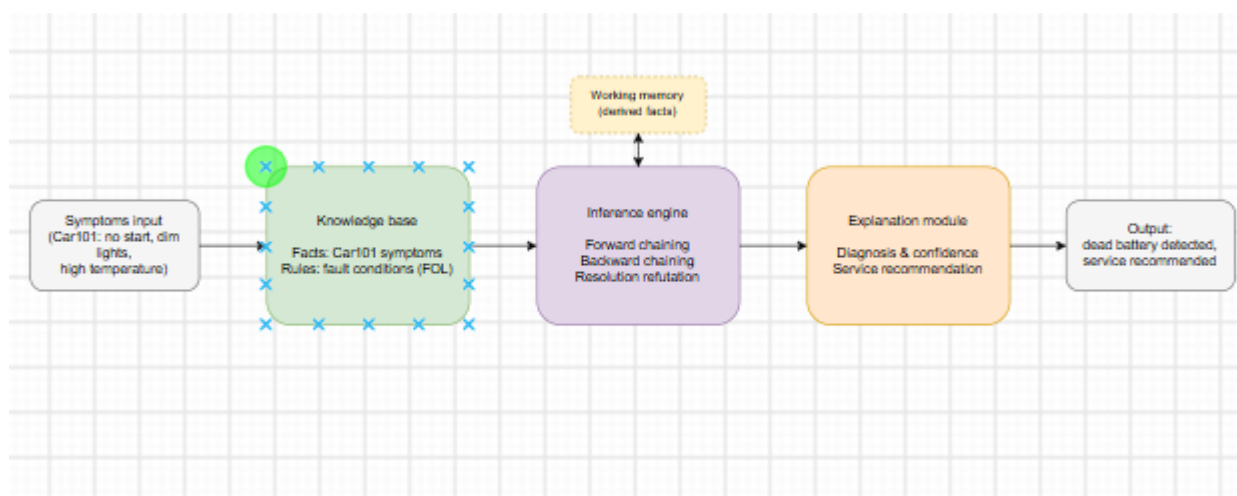


Figure 4 — System architecture: knowledge base, inference engine, and explanation module

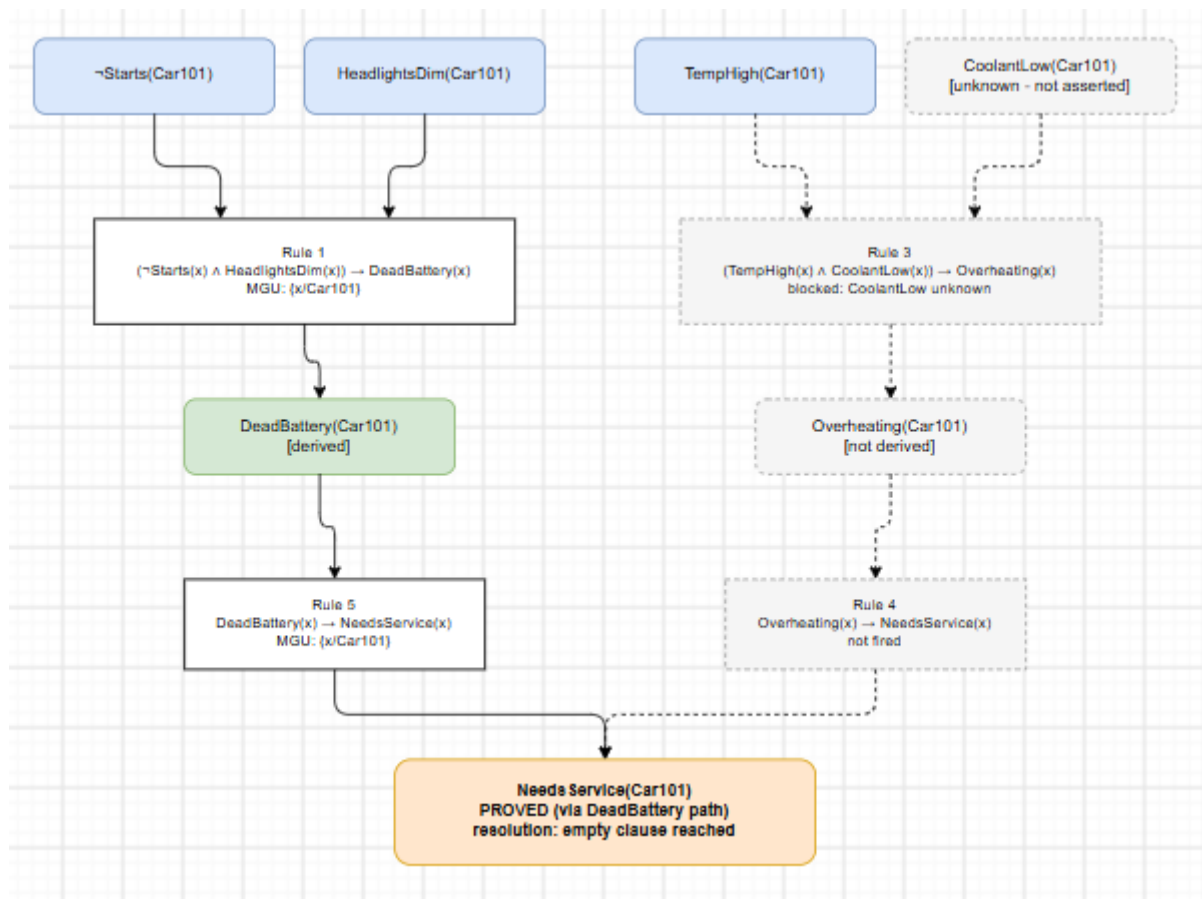


Figure 5 — Knowledge representation network: facts, rules, and derived conclusions for Car101

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
Problem Analysis and Understanding	15%	13–15% – Clearly understands and analyzes the real-world problem, objectives, entities, facts, constraints, and expected inference.	10–12% – Understands the problem with minor omissions.	7–9% – Shows partial understanding with some important elements missing.	0–6% – Poor or incorrect understanding of the problem.
Knowledge Engineering & Knowledge Base Design	15%	13–15% – Develops a complete, consistent, and well-structured knowledge base with appropriate facts, predicates, entities, and rules.	10–12% – Knowledge base is well designed with minor inconsistencies or omissions.	7–9% – Basic knowledge base is developed but several facts/rules are missing or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
Propositional & First-Order Logic Representation	20%	17–20% – Correctly represents the domain using propositions, predicates, variables, constants, quantifiers, and logical connectives with excellent accuracy.	13–16% – Logic representation is mostly correct with minor errors.	9–12% – Provides basic representations but has several logical or syntactic errors.	0–8% – Logic representation is incorrect, incomplete, or missing.
Unification & Resolution	15%	13–15% – Correctly performs substitutions, identifies the MGU, and applies resolution systematically to derive valid conclusions.	10–12% – Applies unification and resolution correctly with minor errors.	7–9% – Demonstrates partial understanding but misses important steps or contains errors.	0–6% – Unable to correctly apply unification or resolution.

Forward Chaining & Backward Chaining	15%	13–15% – Correctly applies both forward and backward chaining with clear step-by-step reasoning and appropriate rule selection.	10–12% – Correctly applies both methods with minor errors or omissions.	7–9% – Demonstrates partial understanding of one or both chaining methods.	0–6% – Incorrectly applies or fails to demonstrate chaining techniques.
Inference & Reasoning Accuracy	10%	9–10% – Produces logically valid conclusions and clearly explains how facts and rules lead to the final inference.	7–8% – Conclusions are mostly correct with minor reasoning gaps.	5–6% – Some correct inferences but reasoning is incomplete or unclear.	0–4% – Conclusions are incorrect, unsupported, or missing.
Knowledge Representation Diagram / System Architecture	5%	5% – Provides a complete, accurate, well-labelled diagram showing knowledge base, inference engine, reasoning process, and output.	4% – Diagram is mostly correct with minor omissions.	3% – Diagram is partially complete or lacks clarity.	0–2% – Diagram is missing or incorrect.
Originality, Critical Thinking & Presentation	5%	5% – Demonstrates excellent independent thinking, appropriate real-world application, comparison of reasoning approaches, and clear presentation.	4% – Demonstrates good reasoning and clear presentation.	3% – Shows limited critical thinking and basic presentation.	0–2% – Shows little originality, weak reasoning, or poor presentation.